OSCAR–GAP interaction

Computational group theory

Silvia Properzi

Vrije Universiteit Brussel

7 May 2026

OSCAR wraps GAP

For group theory, OSCAR wraps the GAP system.

We can always recover the underlying GAP object:

```
1 using Oscar
2
3 G = symmetric_group(5) #is an OSCAR permutation
      group
4
5 H = GapObj(G) #H is the underlying GAP object of G
```

- OSCAR groups are backed by GAP objects
- `GapObj(G)` exposes this representation

OSCAR communicates with GAP via the `GAP.jl` interface.[1]

[1] https://github.com/oscar-system/GAP.jl

How groups are implemented in Oscar

From the OSCAR source code:[2]

```
@attributes mutable struct PermGroup <: GAPGroup
    X::GapObj
    deg::Int64    # G < Sym(deg)
    [...]
end
```

This reflects the idea:

- OSCAR groups are wrappers around GAP objects
- GAP performs the actual computations

[2] https://github.com/oscar-system/Oscar.jl/blob/master/src/Groups/types.jl

What is GAP.jl?

GAP.jl is a low-level interface between Julia and GAP.

It allows Julia to:

- call GAP functions
- access GAP objects
- use GAP packages

Basic objects are converted automatically:

- integers
- strings
- lists
- Booleans

Does NOT implement algebraic objects.

OSCAR and GAP: the architecture

OSCAR is built on top of GAP via a Julia interface layer.

$$\text{OSCAR} \longrightarrow \text{GAP.jl} \longrightarrow \text{GAP}$$

- OSCAR: mathematical interface
- GAP.jl: communication layer (Julia $\leftrightarrow$ GAP)
- GAP: computation engine

How we talk to GAP from OSCAR

1. Stable interface

```
H = GAP.Globals.SmallGroup(8,2)
GAP.Globals.Size(H)
```

2. GAP.jl macro (needed: using GAP)

```
@gap Size(SmallGroup(8,2)) #Pure GAP evaluation
@gap K := SmallGroup(8,2)
K #fails

K = @gap SmallGroup(8,2) #Julia wrapper around a
    GAP object
@gap Size(K)   # fails!
```

How we talk to GAP from OSCAR

3. Evaluate strings (raw GAP execution)

```
GAP.evalstr("L := SmallGroup(8,1)")
@gap Size(L)

L  # fails (Julia does not see GAP variable)
GAP.Globals.Size(L)  # also fails (L is not a
    Julia GapObj)
```

- This runs inside a **separate GAP evaluation context**
- The result is stored in GAP as variable L
- BUT: Julia does not know about L
- `evalstr` does NOT return a Julia object
- it only modifies the internal GAP session

Package loading example

```
GAP.evalstr("LoadPackage(\"YangBaxter\");")
```

Which interface should you use?

GAP.Globals

- explicit, stable, fully controlled
- but verbose (`GAP.Globals.*`)

@gap macro

- cleaner syntax
- good for nested expressions
- still calls GAP under the hood

GAP.evalstr

- executes raw GAP code
- no Julia objects allowed
- mainly for scripts/debugging

Interactive GAP session

You can open a GAP REPL inside Julia:

```
1 GAP.prompt()
```

- full GAP environment
- useful for exploration and debugging
- but results are not automatically captured in Julia

You can also execute GAP files from Julia:

```
1 GAP.Globals.Read("file.g")
```

- runs a GAP script inside the current GAP session
- useful for prewritten computations or package code

From GAP to OSCAR

Key fact:

```
permutation_group(G)
```

- Works if G is a GAP permutation group
- Returns an OSCAR permutation group

Example

```
G = GAP.Globals.SymmetricGroup(5)
OscarG = permutation_group(G)
order(OscarG)
```

Other group types (GAP-side objects):
The same works for `pc_group` and `fp_group`

Load package

```
using GAP

GAP.Globals.LoadPackage(GAP.Obj("YangBaxter"))
```

Alternative:

```
GAP.evalstr("LoadPackage(\"YangBaxter\");")
```

Back to OSCAR

```
X = GAP.Globals.SmallIYB(4,2)

G = GAP.Globals.YBPermutationGroup(X)
```

```
using Oscar

OscarG = permutation_group(G)

order(OscarG)
```

```
S = GAP.Globals.StructureGroup(X)
```

Wrappers

We can also define our own wrappers:

```
GAP.@wrap SmallIYB(x::GapInt,y::GapInt)::GapObj
GAP.@wrap YBPermutationGroup(x::GapObj)::GapObj

X=SmallIYB(5,2)
permutation_group(YBPermutationGroup(X))
function YB_permutation_group(X::GapObj)
return permutation_group(YBPermutationGroup(X))
end
```